ALARD UNIVERSITY

B.Tech CSF

OPERATING SYSTEMS
MINI PROJECT REPORT

PAGE REPLACEMENT SIMULATOR

Student Name	Ketan Patil
PRN	26ET2052003
Roll No.	15
Branch	B.Tech CSF
Subject	Operating Systems

Submitted for academic evaluation

CERTIFICATE

This is to certify that the mini project titled “Page Replacement Simulator” has been prepared by Ketan Patil (PRN: 26ET2052003, Roll No.: 15), B.Tech CSF, for the Operating Systems subject. The project demonstrates FIFO and LRU page replacement algorithms.

ACKNOWLEDGEMENT

I sincerely thank my subject faculty for guidance and support during this mini project. I am also thankful to Alard University for providing the opportunity to study and apply Operating Systems concepts.

ABSTRACT

A Page Replacement Simulator demonstrates how an operating system manages pages when requested pages cannot all fit in the available memory frames. This project implements FIFO (First-In, First-Out) and LRU (Least Recently Used), counts page faults, and compares the results.

1. INTRODUCTION

An operating system manages memory for running programs. In virtual memory, data is divided into pages and physical memory is divided into frames. When a requested page is not in memory, a page fault occurs. If all frames are occupied, a page replacement algorithm chooses which page to remove.

2. PROBLEM STATEMENT

Create a program that accepts a page-reference string and frame count, simulates page replacement using FIFO and LRU, and displays the total page faults for both algorithms.

3. OBJECTIVES

- Understand pages, frames, and page faults.
- Implement FIFO page replacement.
- Implement LRU page replacement.
- Compare the page-fault totals for both algorithms.

4. SYSTEM REQUIREMENTS

Hardware: Computer or laptop.
Software: C++ compiler such as GCC, Code::Blocks, or Visual Studio.
Language: C++.
Input: Page-reference sequence and number of frames.
Output: FIFO and LRU page-fault totals.

5. THEORY

Page fault: A requested page is not currently in main memory.
Frame: A fixed-size block of physical memory that holds one page.
FIFO: Replaces the page that entered memory first.
LRU: Replaces the page that has not been used for the longest time recently.

6. ALGORITHM / METHODOLOGY

FIFO: Check each requested page. If it is absent, count a fault; use an empty frame if available, otherwise replace the oldest page. LRU: Check each requested page. If present, update its recent-use time. If

absent, count a fault and use an empty frame or replace the least recently used page.

7. SOURCE CODE (C++)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int fifoFaults(const vector<int>& pages, int cap) {
    vector<int> frames;
    int faults = 0, next = 0;
    for (int p : pages) {
        if (find(frames.begin(), frames.end(), p) != frames.end())
            continue;
        faults++;
        if ((int)frames.size() < cap) frames.push_back(p);
        else { frames[next] = p; next = (next + 1) % cap; }
    }
    return faults;
}

int lruFaults(const vector<int>& pages, int cap) {
    vector<int> frames, lastUsed;
    int faults = 0;
    for (int i = 0; i < (int)pages.size(); i++) {
        int p = pages[i];
        auto it = find(frames.begin(), frames.end(), p);
        if (it != frames.end()) lastUsed[it - frames.begin()] = i;
        else {
            faults++;
            if ((int)frames.size() < cap) {
                frames.push_back(p); lastUsed.push_back(i);
            } else {
                int j = min_element(lastUsed.begin(), lastUsed.end()) - lastUsed.begin();
                frames[j] = p; lastUsed[j] = i;
            }
        }
    }
    return faults;
}

int main() {
    int n, cap;
    cout << "Enter number of page references: ";
    cin >> n;
    if (n <= 0) { cout << "Invalid number of references\\n"; return 0; }

    vector<int> pages(n);
    cout << "Enter page reference string: ";
    for (int i = 0; i < n; i++) cin >> pages[i];

    cout << "Enter number of frames: ";
    cin >> cap;
    if (cap <= 0) { cout << "Invalid number of frames\\n"; return 0; }

    cout << "FIFO page faults: " << fifoFaults(pages, cap) << endl;
    cout << "LRU page faults: " << lruFaults(pages, cap) << endl;
    return 0;
}
```

8. SAMPLE INPUT AND OUTPUT

```
Enter number of page references: 12
Enter page reference string: 1 2 3 1 4 5 2 1 2 3 4 5
Enter number of frames: 3
FIFO page faults: 10
LRU page faults: 10
```

9. RESULT AND DISCUSSION

The simulator demonstrates that FIFO and LRU use different replacement rules. The page-fault count depends on the reference string and the number of frames. For this sample input, both algorithms produce 10 page faults.

10. CONCLUSION AND FUTURE SCOPE

The project provides a simple way to understand page faults and memory management. Future improvements may include Optimal page replacement, step-by-step frame display, charts, and a graphical user interface.

REFERENCES

• Operating System Concepts — Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
• Operating Systems: Three Easy Pieces — Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau.
• Classroom notes on virtual memory and page replacement.